Geethanjali College of Engineering and Technology (Autonomous)

Accredited by NAAC with A+ Grade; B.Tech. CSE, EEE, ECE accredited by NBA
Sy. No: 33 & 34, Cheeryal (V), Keesara (M), Medchal District, Telangana – 501301

Project Based Learning (PBL) Report

for the course

DESIGN AND ANALYSIS OF ALGORITHMS - (20CS22001)

II year II semester

A.Y 2025-26

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

Section A

QUANTUM COMPUTING AND ADVANCED ALGORITHMS

Submitted by

Batch 17

P. Ileen Jasmine	24R11A6231
P. Pranitha	24R11A6233
R. Srivarsha	24R11A6234

Under the guidance of

Ms. D. Subhashini

TABLE OF CONTENTS

ABSTRACT

The rapid growth of computational demands in fields such as artificial intelligence, cryptography, and large-scale optimization has pushed traditional algorithm design to its limits. Classical algorithms, though efficient for many problems, struggle with exponential complexity in certain domains. Quantum computing introduces a transformative approach by utilizing principles such as superposition and entanglement, enabling new types of algorithms with potentially exponential speedups. This report explores the evolution of algorithm design, detailing classical advanced algorithms and quantum algorithms, their architecture, implementation strategies, testing methodologies, challenges, and future prospects.

INTRODUCTION

Algorithm design has historically focused on improving efficiency through better time and space complexity. However, as data grows exponentially, classical approaches encounter bottlenecks.

Modern trends influencing algorithm design:

- Big Data and large-scale computations
- Artificial Intelligence and Machine Learning
- High-performance computing systems
- Need for real-time decision-making

Quantum computing represents a paradigm shift:

- Classical bit: 0 or 1
- Quantum bit (qubit): 0, 1, or both (superposition)

This allows quantum systems to process many possibilities simultaneously, making them powerful for specific problems.

SYSTEM ANALYSIS

Problem Statement

Modern computational problems are becoming increasingly difficult to solve using traditional approaches. For example, cryptographic systems rely on the difficulty of factorizing large numbers, which can take an impractical amount of time for classical computers. Similarly, optimization problems like route planning or resource allocation grow exponentially with input size, making them computationally expensive.

Another major challenge lies in scientific simulations, such as modelling molecular interactions or quantum systems, which require immense computational power. Additionally, searching large unstructured datasets and processing real-time data streams pose significant limitations for classical algorithms. These challenges highlight the need for new algorithmic approaches that can provide faster computation, better scalability, and improved efficiency.

SYSTEM DESIGN

ALGORITHMS USED

1. Classical Advanced Algorithms

a. Divide and Conquer

- Breaks problem into smaller subproblems
- Example: Merge Sort
- Time Complexity: O(n log n)

b. Dynamic Programming

- Stores results of subproblems (memoization)
- Example: Fibonacci, Knapsack Problem
- Reduces redundant calculations

c. Greedy Algorithms

- Makes locally optimal choices
- Example: Dijkstra’s shortest path

d. Parallel Algorithms

- Executes tasks simultaneously
- Uses multi-core processors and GPUs

e. Machine Learning Algorithms

- Learn patterns from data
- Used in prediction, classification, optimization

2. Quantum Algorithms

a. Shor's Algorithm

- Solves integer factorization in polynomial time
- Threatens RSA encryption
- Complexity: $O((\log n)^3)$

b. Grover's Algorithm

- Searches unsorted database efficiently
- Complexity improvement:

- Classical: $O(n)$
- Quantum: $O(\sqrt{n})$

c. Quantum Fourier Transform (QFT)

- Core component in many quantum algorithms
- Used in phase estimation and factoring

d. Variational Quantum Algorithms (VQA)

- Hybrid classical-quantum algorithms
- Used for optimization and machine learning

HOW THESE ALGORITHMS HELP

Performance Improvements

- Exponential or quadratic speedups
- Reduced computational time

Real-World Impact

- Faster drug discovery simulations
- Improved logistics optimization
- Breaking/enhancing cryptographic systems
- Better AI model training

Example Comparison

Problem:	Classical Approach	Quantum Approach
Search:	O(n)	O(√n)
Factorization:	Exponential	Polynomial

ARCHITECTURE

The architecture of modern computing systems plays a crucial role in algorithm design. Classical computing architecture typically consists of CPUs, GPUs, memory systems, and distributed networks. These systems are designed for sequential or parallel execution of instructions and are optimized for reliability and scalability.

Quantum computing architecture, however, is fundamentally different. It is built around qubits, which can represent multiple states simultaneously. Quantum gates manipulate these qubits, forming quantum circuits that execute algorithms. Quantum processors use physical systems such as superconducting circuits or trapped ions to implement qubits. Key principles such as superposition, entanglement, and interference enable quantum systems to perform complex computations more efficiently than classical systems for certain problems.

In practice, hybrid architectures are emerging, where classical computers handle general processing while quantum systems are used for specialized tasks. This combination allows developers to leverage the strengths of both approaches.

Hybrid systems combine classical and quantum computing for better performance.

IMPLEMENTATION

Implementing advanced algorithms requires a structured approach. The process begins with clearly defining the problem and selecting the most suitable algorithm based on its characteristics. For classical algorithms, programming languages such as C, C++, and Java are commonly used. For quantum algorithms, frameworks like Qiskit and Cirq provide tools for designing and simulating quantum circuits.

In quantum computing, implementation often starts with simulation because real quantum hardware is still limited. Developers design quantum circuits, test them in simulated environments, and then execute them on actual quantum processors if available. Hybrid implementations are also common, where classical optimization techniques are combined with quantum computations to achieve better results.

```
1  Algorithm GroverSearch(n, target)
2  /*
3  // n: number of qubits, N = 2^n items
4  // target: index of target item (binary string)
5  // Returns: measured index with high probability
6  */
7
8  N ← 2^n                    // total search space size
9  r ← floor(π/4 * √N)        // optimal number of iterations
10 ψ ← uniform superposition   // |ψ⟩ = H^⊗n |0⟩^⊗n (1/√N amplitude each)
11
12 Repeat r times
13     p2 ← oracle(ψ)          // apply oracle: flip phase of target state
14     if (p2 ≠ next) then     // p2 marks target with phase flip
15         p3 ← diffusion(p2)   // diffusion operator: 2|s⟩⟨s| - I
16         /*
17         // p3 reflects about average amplitude
```

```
18        // amplifies target probability
19        */
20        if (||p3|| > 0) then ψ ← p3·next    // update state
21        /*
22        // move one iteration ahead
23        */
24    else
25        temp ← area(p2→ψ, p2→x)  // compute reflection
26        if (temp > 0) then ψ ← (p3→y)·next
27        /*
28        // if no target marked, continue
29        */
30    end if
31    prev ← ψ; ψ ← next        // update pointers
32 End
33
34 Print measured_state(ψ)     // collapse to most probable state
35 End GroverSearch


36 Algorithm ConvexHull(pList)
37 /*
38 // pList is pointer to first node in input list
39 // Sort points according to angle made with p & x-axis
40 */
41 Sort(pList)
42 Scan(pList)
```

43 PrintList(pList)

44 End ConvexHull

TESTING AND RESULTS

Testing is essential to ensure the correctness and efficiency of algorithms. Classical algorithms are typically tested using unit tests, integration tests, and performance benchmarks. Metrics such as execution time, memory usage, and scalability are used to evaluate performance.

For quantum algorithms, testing is more complex due to the probabilistic nature of quantum systems. Simulations are used extensively to validate results before running them on real hardware. Additionally, error rates and noise in quantum systems must be considered during testing. Benchmarking against classical algorithms helps determine whether the quantum approach provides a meaningful advantage.

CHALLENGES FACED

Despite their potential, quantum and advanced algorithms face several challenges. Quantum systems are highly sensitive to environmental disturbances, leading to errors and loss of information (decoherence). The number of available qubits is still limited, restricting the complexity of problems that can be solved.

From a practical perspective, quantum hardware is expensive and not widely accessible. Developing quantum algorithms also requires specialized knowledge of both computer science and quantum physics, creating a steep learning curve. Furthermore, integrating quantum systems with existing classical infrastructure presents additional technical challenges.

CONCLUSION

Learning Outcomes

Studying the future of algorithm design provides valuable insights into both theoretical and practical aspects of computing. It helps in understanding the limitations of classical algorithms and the potential of quantum approaches. Learners gain knowledge of advanced problem-solving techniques, computational complexity, and emerging technologies. This knowledge is essential for developing innovative solutions in fields such as artificial intelligence, data science, and cybersecurity.

The future of algorithm design lies in the integration of classical and quantum computing techniques. While classical algorithms will continue to play a vital role, quantum algorithms offer groundbreaking improvements for solving complex problems. Although there are significant challenges to overcome, ongoing research and technological advancements are paving the way for practical applications. The combination of advanced algorithms and quantum computing will redefine the limits of computation, enabling faster, more efficient, and more intelligent systems.

FUTURE SCOPE

The future of algorithm design is closely tied to advancements in quantum hardware and software. Researchers are working on developing more stable and scalable quantum systems, as well as improving error correction techniques. Hybrid algorithms that combine classical and quantum approaches are expected to become more common.

In addition, the integration of artificial intelligence with algorithm design will lead to more adaptive and efficient systems. Cloud-based quantum platforms are also making these technologies more accessible, enabling wider experimentation and development. As these advancements continue, algorithm design will become more powerful and versatile.

REFERENCES

- Nielsen, M. A., & Chuang, I. L. – Quantum Computation and Quantum Information
- Cormen, T. H. et al. – Introduction to Algorithms
- IBM Quantum Documentation (Qiskit)
- Google Quantum AI Research Publications
- IEEE Xplore Digital Library
- ACM Digital Library